

# Cognizant Digital Nurture 5.0

Deep Skilling Program

Week 7 – DevOps & Cloud

CI/CD Pipeline Fundamentals

---

**Prepared By**

**Ashreen Fathima**

B.E. Computer Science and Design

RMK Engineering College

Email: ashreenfathima26@gmail.com

Academic Year: 2026–2027

---

## Table of Contents

1. Introduction
  2. What is CI/CD?
  3. Why CI/CD?
  4. Traditional Development vs CI/CD
  5. Continuous Integration (CI)
  6. Continuous Delivery (CD)
  7. Continuous Deployment
  8. CI/CD Pipeline Stages
  9. CI/CD Tools
  10. CI/CD Workflow
  11. Benefits of CI/CD
  12. Best Practices
  13. Conclusion
- 

## 1. Introduction

Modern software development requires rapid delivery of high-quality applications while maintaining reliability and minimizing deployment risks. Manual build, testing, and deployment processes are often time-consuming and prone to human errors. Continuous Integration and Continuous Delivery/Deployment (CI/CD) automate these processes, enabling teams to deliver software efficiently and consistently.

CI/CD has become an essential DevOps practice, supporting agile development, automation, and faster software releases.

---

2. What is CI/CD?

CI/CD stands for **Continuous Integration** and **Continuous Delivery/Continuous Deployment**. It is a set of practices that automate the process of integrating code changes, testing applications, and deploying software to production environments.

Definition

CI/CD is a DevOps methodology that automates software building, testing, integration, and deployment to enable faster, reliable, and continuous software delivery.

---

3. Why CI/CD?

Organizations implement CI/CD to improve software quality and accelerate delivery.

Key objectives include:

- Faster software releases
  - Automated build process
  - Continuous testing
  - Reduced manual effort
  - Early bug detection
  - Consistent deployments
  - Improved collaboration
  - Better customer satisfaction
- 

4. Traditional Development vs CI/CD

| Traditional Development     | CI/CD Approach         |
|-----------------------------|------------------------|
| Manual builds               | Automated builds       |
| Manual testing              | Automated testing      |
| Slow deployments            | Fast deployments       |
| Frequent integration issues | Continuous integration |
| High deployment risk        | Reliable deployments   |
| Longer release cycles       | Frequent releases      |

---

## 5. Continuous Integration (CI)

Continuous Integration is the practice of automatically integrating code changes into a shared repository multiple times a day.

### Key Activities

- Code Commit
- Source Code Management
- Automated Build
- Unit Testing
- Code Quality Checks

### Benefits

- Early detection of defects
- Improved code quality
- Faster feedback
- Reduced integration conflicts

---

## 6. Continuous Delivery (CD)

Continuous Delivery ensures that every successful build is automatically prepared for deployment.

In this approach, applications undergo automated testing and validation before being released to production with manual approval.

### Features

- Automated testing
- Release readiness
- Reliable deployments
- Reduced deployment risk

---

## 7. Continuous Deployment

Continuous Deployment automatically releases every validated build directly into the production environment without manual intervention.

### Features

- Fully automated deployment
- Faster software delivery

- Immediate feature availability
- Continuous customer feedback

---

## Difference Between Continuous Delivery and Continuous Deployment

### Continuous Delivery

Manual approval before production

Deployment is optional

Suitable for controlled releases

### Continuous Deployment

Automatic deployment to production

Deployment is automatic

Suitable for rapid release environments

---

## 8. CI/CD Pipeline Stages

A typical CI/CD pipeline consists of the following stages:

Developer

|

Code Commit

|

Source Control (Git)

|

Build

|

Automated Testing

|

Code Quality Analysis

|

Package Application

|

Deploy to Staging

|

Approval (Optional)

|

Deploy to Production

|

Monitoring

Pipeline Stages

| Stage         | Description                                                   |
|---------------|---------------------------------------------------------------|
| Code Commit   | Developers push code changes to the repository                |
| Build         | Application source code is compiled                           |
| Test          | Automated tests validate application functionality            |
| Code Analysis | Static code analysis and quality checks                       |
| Package       | Application artifacts are generated                           |
| Deploy        | Application is deployed to staging or production              |
| Monitor       | Application health and performance are continuously monitored |

---

9. CI/CD Tools

Various tools support CI/CD implementation.

| Category                | Popular Tools                      |
|-------------------------|------------------------------------|
| Version Control         | Git, GitHub                        |
| Build Automation        | Maven, Gradle                      |
| CI/CD Platforms         | Jenkins, GitHub Actions, GitLab CI |
| Containerization        | Docker                             |
| Container Orchestration | Kubernetes                         |
| Code Quality            | SonarQube                          |
| Monitoring              | Prometheus, Grafana                |

---

10. CI/CD Workflow

Developer

|



Git Repository

|



Continuous Integration

(Build & Test)



Continuous Delivery

(Deploy to Staging)



Approval (Optional)



Continuous Deployment

(Production)



Monitoring & Feedback

---

## 11. Benefits of CI/CD

CI/CD provides numerous advantages to software development teams.

- Faster software delivery
  - Improved software quality
  - Reduced deployment failures
  - Faster issue detection
  - Increased development productivity
  - Consistent release process
  - Better collaboration
  - Enhanced customer experience
  - Lower operational costs
  - Rapid feedback cycle
-

## 12. Best Practices

To implement CI/CD effectively:

- Commit code frequently.
- Automate build and testing processes.
- Maintain version control using Git.
- Perform automated code quality checks.
- Keep deployments small and incremental.
- Monitor deployed applications continuously.
- Secure the CI/CD pipeline.
- Regularly update automation tools.

---

## 13. Conclusion

CI/CD is a fundamental DevOps practice that enables organizations to automate software integration, testing, and deployment processes. By reducing manual effort and ensuring consistent delivery, CI/CD improves software quality, accelerates release cycles, and enhances collaboration between development and operations teams. Implementing CI/CD is essential for modern software engineering and cloud-native application development.